

Relatório Técnico: Implementação de Pipeline RISC-V

Arthur Carvalho Rodrigues, Débora Luiza de Paula Silva,
Gustavo Henrique Rodrigues de Castro e Mariana Almeida Mendonça

24 de maio de 2026

1 Introdução

colocar a introdução e o resto aqui

2 Implementação - contiuarrrrrr

Foi criado um módulo de topo, denominado `cache_top`, responsável por integrar o controlador de cache e a memória principal. Esse módulo recebe os sinais da CPU, repassa as requisições ao controlador de cache e conecta internamente a interface entre o controlador e a memória principal.

Essa separação torna o projeto mais modular, pois o `cache_controller` contém apenas a lógica de controle da cache, o `main_memory` modela a memória principal com latência, e o `cache_top` realiza a integração entre os dois módulos. Dessa forma, o *testbench* pode validar o sistema completo a partir da interface da CPU.

3 Depuração de erro temporal entre cache e memória principal

Durante a etapa de validação funcional do controlador de cache, foram inicialmente executados testes básicos de leitura e escrita. Esses testes verificaram corretamente os seguintes cenários:

- leitura com miss;
- leitura com hit;
- escrita com hit;
- leitura após escrita;
- escrita com miss utilizando *write-allocate*;
- leitura após *write-allocate*.

A simulação inicial apresentou resultados coerentes para esses casos. Por exemplo, uma leitura no endereço `8'h10` resultou inicialmente em miss e retornou o valor `32'h1004`. Em seguida, uma nova leitura no mesmo endereço resultou em hit, confirmando que o bloco havia sido carregado corretamente para a cache. Também foi validado que uma escrita com hit atualizava o valor armazenado na cache e que leituras posteriores retornavam o valor atualizado.

3.1 Identificação do problema

Após a validação dos casos básicos, foram adicionados testes envolvendo conflitos de índice, substituição de blocos e escrita de volta para a memória principal. Esses testes utilizam endereços diferentes que mapeiam para o mesmo conjunto da cache, permitindo avaliar o comportamento da política de substituição LRU e da política de escrita *write-back*.

A divisão de endereço adotada no projeto foi:

- `addr[7:4]`: campo de tag;
- `addr[3:2]`: campo de índice;
- `addr[1:0]`: campo de offset.

Dessa forma, os endereços `8'h08`, `8'h18` e `8'h28` possuem o mesmo campo de índice e, portanto, mapeiam para o mesmo conjunto da cache.

Como a memória principal foi inicializada com a regra:

```
memory_array[i] = 32'h1000 + i;
```

os valores esperados para esses endereços eram:

Endereço	Palavra acessada	Valor esperado
<code>8'h08</code>	2	<code>32'h1002</code>
<code>8'h18</code>	6	<code>32'h1006</code>
<code>8'h28</code>	10	<code>32'h100A</code>

Tabela 1: Valores esperados para acessos que mapeiam para o mesmo conjunto da cache.

No entanto, durante a simulação, foi observado o seguinte comportamento:

```
READ | Addr: 8'h08 | Data: 32'h1002 | Hit: 0
READ | Addr: 8'h18 | Data: 32'h1002 | Hit: 0
```

O primeiro acesso estava correto, pois o endereço `8'h08` corresponde à palavra 2 da memória. Entretanto, o segundo acesso estava incorreto, pois o endereço `8'h18` deveria retornar `32'h1006`, mas retornou novamente `32'h1002`. Isso indicou que o controlador estava armazenando ou retornando um valor antigo de `mem_rdata`.

3.2 Causa do erro

A causa do problema estava relacionada ao sincronismo entre os sinais `mem_ready` e `mem_rdata`. A memória principal foi modelada com latência, de modo que, ao final de uma operação de leitura, ela atualiza o dado de saída e sinaliza que a operação foi concluída.

A lógica da memória principal utiliza atribuições não bloqueantes:

```
mem_rdata <= memory_array[saved_word_index];  
mem_ready <= 1'b1;
```

Em SystemVerilog, dentro de blocos sequenciais, atribuições não bloqueantes não atualizam imediatamente os sinais. Os novos valores só ficam disponíveis após a borda de clock atual.

Na versão inicial do controlador, a cache era atualizada no mesmo ciclo em que o sinal `mem_ready` era detectado:

```
if (mem_ready) begin  
    data[req_index][victim_way] <= mem_rdata;  
    cpu_rdata <= mem_rdata;  
end
```

Com isso, o controlador lia o valor anterior de `mem_rdata`, e não o valor recém-produzido pela memória principal. Esse erro se tornou evidente nos testes com misses consecutivos, nos quais o dado retornado por um acesso correspondia ao acesso anterior.

3.3 Correção adotada

Para corrigir o problema, foi adicionado um estado intermediário à máquina de estados do controlador de cache. O fluxo anterior era:

ALLOCATE -> RESPOND

Após a correção, o fluxo passou a ser:

ALLOCATE -> UPDATE_CACHE -> RESPOND

O estado **ALLOCATE** passou a ser responsável apenas por solicitar a leitura da memória principal e aguardar o sinal `mem_ready`. Quando `mem_ready` é ativado, a máquina de estados avança para o estado **UPDATE_CACHE**. Nesse estado, o valor de `mem_rdata` já está estável e pode ser corretamente armazenado na cache.

O comportamento corrigido pode ser resumido da seguinte forma:

1. No estado **ALLOCATE**, o controlador solicita a leitura da memória principal.
2. A memória principal, após sua latência interna, ativa `mem_ready`.
3. O controlador avança para o estado **UPDATE_CACHE**.
4. No estado **UPDATE_CACHE**, o dado presente em `mem_rdata` é gravado na cache.
5. A tag, o bit `valid`, o bit `dirty` e a informação de LRU são atualizados.
6. No estado **RESPOND**, o controlador sinaliza `cpu_ready`.

Essa modificação garante que a cache não seja atualizada com dados atrasados da memória principal.

3.4 Importância da correção

Esse erro reforçou a importância de considerar o comportamento temporal dos sinais em projetos de hardware. Diferentemente de um programa sequencial em software, em uma descrição de hardware os sinais atualizados em blocos sequenciais não assumem seus novos valores instantaneamente.

Portanto, mesmo que o sinal `mem_ready` indique que a memória terminou uma operação, o controlador precisa garantir que o valor de `mem_rdata` esteja estável antes de armazená-lo na cache.

A inclusão do estado `UPDATE_CACHE` tornou a máquina de estados mais robusta e mais coerente com o funcionamento síncrono dos módulos do projeto. Além disso, a falha foi identificada somente após a criação de testes mais completos, o que evidencia a importância de validar não apenas os casos básicos de hit e miss, mas também os cenários de conflito de índice, substituição de blocos e escrita de volta para a memória principal.

4 Resultados e Validação Funcional

Após a implementação do controlador de cache, foram realizados testes funcionais por meio de um *testbench* em SystemVerilog. O objetivo desses testes foi validar o comportamento da cache em operações de leitura, escrita, tratamento de misses, política de substituição LRU, política *write-back* e política *write-allocate*.

A simulação foi executada utilizando o Icarus Verilog. O arquivo `wave.vcd` também foi gerado para permitir a visualização das formas de onda no GTKWave.

4.1 Testes realizados

A Tabela ?? resume os principais testes realizados e os respectivos resultados obtidos.

4.2 Validação de leitura com miss e hit

Inicialmente, foi realizada uma leitura no endereço `8'h10`. Como a cache estava vazia após o reset, o acesso resultou em miss. O controlador buscou o dado correspondente na memória principal, carregou o bloco na cache e retornou o valor `32'h1004`.

Em seguida, uma nova leitura para o mesmo endereço foi realizada. Dessa vez, o dado já estava presente na cache, resultando em hit.

```
READ | Addr: 8'h10 | Data: 32'h1004 | Hit: 0
READ | Addr: 8'h10 | Data: 32'h1004 | Hit: 1
```

Esse resultado confirma o funcionamento correto do caminho de leitura, tanto para miss quanto para hit.

4.3 Validação de escrita com hit e política write-back

Após o carregamento do endereço `8'h10` na cache, foi realizada uma escrita nesse mesmo endereço com o valor `32'hdeadbeef`. Como o bloco já estava presente na cache, a operação resultou em hit. O dado foi atualizado na cache e a linha foi marcada como *dirty*, conforme a política *write-back*.

Uma leitura posterior no mesmo endereço retornou o valor atualizado:

Teste	Cenário validado	Resultado
1	Leitura com miss e alocação do bloco na cache	Passou
2	Leitura com hit após o bloco já estar na cache	Passou
3	Escrita com hit utilizando política <i>write-back</i>	Passou
4	Leitura após escrita, confirmando atualização do dado na cache	Passou
5	Escrita com miss utilizando política <i>write-allocate</i>	Passou
6	Leitura após <i>write-allocate</i> , confirmando que o bloco foi carregado na cache	Passou
7	Conflito de índice entre endereços que mapeiam para o mesmo conjunto	Passou
8	Atualização da informação de LRU após novo acesso a uma via	Passou
9	Substituição de bloco seguindo a política LRU	Passou
10	Confirmação da substituição LRU por meio de novos acessos	Passou
11	Escrita de volta de bloco marcado como <i>dirty</i>	Passou
12	Verificação de que a memória principal foi atualizada após o <i>write-back</i>	Passou

Tabela 2: Resumo dos testes funcionais realizados no controlador de cache.

WRITE | Addr: 8'h10 | Data: 32'hdeadbeef | Hit: 1
 READ | Addr: 8'h10 | Data: 32'hdeadbeef | Hit: 1

Esse teste confirma que a escrita com hit atualiza corretamente a cache sem escrever imediatamente na memória principal.

4.4 Validação de escrita com miss e política *write-allocate*

Também foi testada uma escrita em um endereço ainda não presente na cache, 8'h24, com o valor 32'hcafebabe. Como o bloco não estava na cache, ocorreu um miss. Pela política *write-allocate*, o bloco foi alocado na cache e, em seguida, o valor da escrita foi armazenado.

A leitura posterior do mesmo endereço resultou em hit e retornou o valor escrito:

WRITE | Addr: 8'h24 | Data: 32'hcafebabe | Hit: 0
 READ | Addr: 8'h24 | Data: 32'hcafebabe | Hit: 1

Esse resultado confirma que a política *write-allocate* foi implementada corretamente.

4.5 Validação de conflitos de índice e substituição LRU

Para validar a política de substituição, foram utilizados endereços que mapeiam para o mesmo conjunto da cache. Considerando a divisão de endereço adotada:

- `addr[7:4]`: tag;
- `addr[3:2]`: índice;
- `addr[1:0]`: offset.

Os endereços `8'h08`, `8'h18` e `8'h28` possuem o mesmo campo de índice e, portanto, mapeiam para o mesmo conjunto.

Primeiro, os endereços `8'h08` e `8'h18` foram acessados, preenchendo as duas vias do conjunto. Em seguida, o endereço `8'h08` foi acessado novamente, atualizando a informação de LRU e tornando `8'h18` o bloco menos recentemente usado. Ao acessar `8'h28`, ocorreu um miss e o controlador substituiu o bloco correspondente a `8'h18`.

A sequência observada foi:

```
READ | Addr: 8'h08 | Data: 32'h1002 | Hit: 0
READ | Addr: 8'h18 | Data: 32'h1006 | Hit: 0
READ | Addr: 8'h08 | Data: 32'h1002 | Hit: 1
READ | Addr: 8'h28 | Data: 32'h100a | Hit: 0
READ | Addr: 8'h08 | Data: 32'h1002 | Hit: 1
READ | Addr: 8'h18 | Data: 32'h1006 | Hit: 0
```

O último acesso a `8'h18` resultou em miss, confirmando que esse foi o bloco substituído. Isso valida o funcionamento da política LRU simples adotada no projeto.

4.6 Validação de write-back com bloco dirty

Por fim, foi validado o comportamento de escrita de volta para a memória principal. Para isso, foi realizada uma escrita no endereço `8'h0C` com o valor `32'h11111111`. Como o endereço não estava na cache, ocorreu um miss, seguido de alocação do bloco e atualização da linha, que passou a ser marcada como *dirty*.

Em seguida, foram realizados acessos a outros endereços que mapeiam para o mesmo conjunto, forçando a substituição do bloco correspondente a `8'h0C`. Como esse bloco estava marcado como *dirty*, o controlador precisou escrever seu conteúdo de volta na memória principal antes de substituí-lo.

A sequência relevante foi:

```
WRITE | Addr: 8'h0c | Data: 32'h11111111 | Hit: 0
READ  | Addr: 8'h1c | Data: 32'h1007      | Hit: 0
READ  | Addr: 8'h1c | Data: 32'h1007      | Hit: 1
READ  | Addr: 8'h2c | Data: 32'h100b      | Hit: 0
READ  | Addr: 8'h0c | Data: 32'h11111111 | Hit: 0
```

O último acesso ao endereço `8'h0C` resultou em miss, pois o bloco havia sido substituído. Entretanto, o valor retornado foi `32'h11111111`, confirmando que o dado modificado foi corretamente escrito de volta na memória principal durante o processo de substituição.

Esse teste valida o funcionamento conjunto da política *write-back*, do bit *dirty* e da substituição de blocos.

4.7 Conclusão da validação

Os testes realizados cobriram os principais comportamentos esperados do controlador de cache: leitura com hit e miss, escrita com hit e miss, alocação em miss de escrita, substituição por LRU, conflitos de índice e escrita de volta de blocos modificados. Todos os testes apresentaram os resultados esperados, indicando que o controlador implementado atende à especificação definida para o projeto.

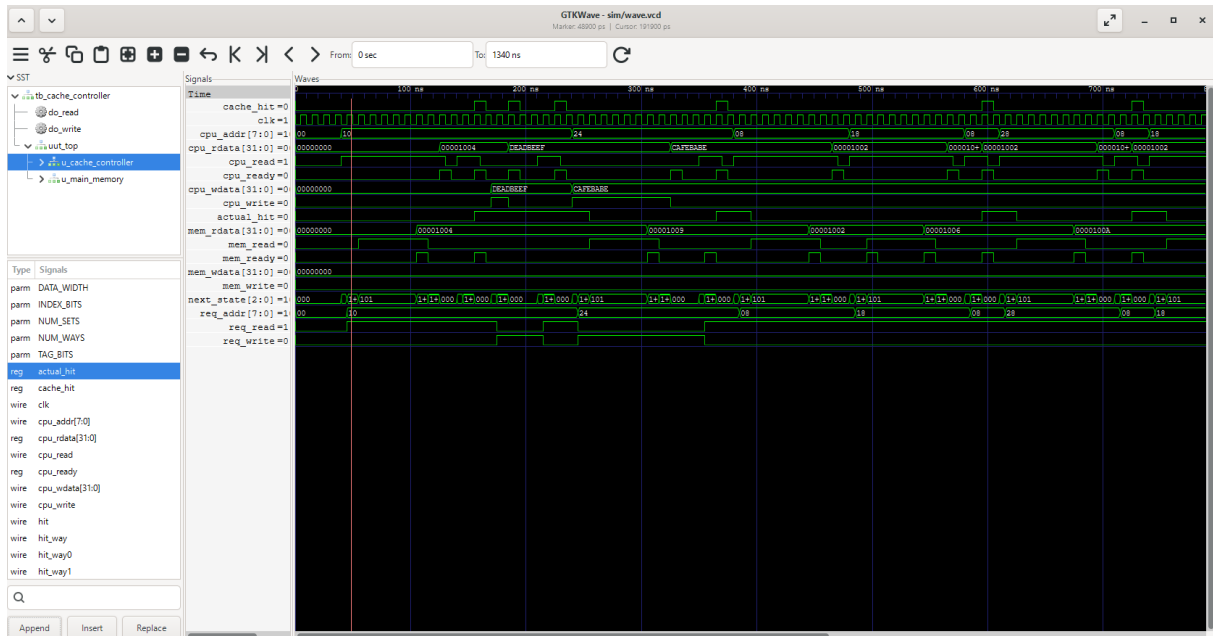


Figura 1: Visão geral da simulação do controlador de cache, mostrando operações de leitura, escrita, misses, hits, substituição LRU e write-back.

A Figura ?? apresenta uma visão geral da simulação. É possível observar a sequência de acessos realizados pelo testbench, com variação dos sinais `cpu_addr`, `cpu_rdata`, `cache_hit`, `mem_read`, `mem_write` e `cpu_ready`. Os pulsos de `mem_read` ocorrem nos acessos com miss, enquanto os pulsos de `mem_write` aparecem durante a escrita de volta de um bloco marcado como *dirty*.

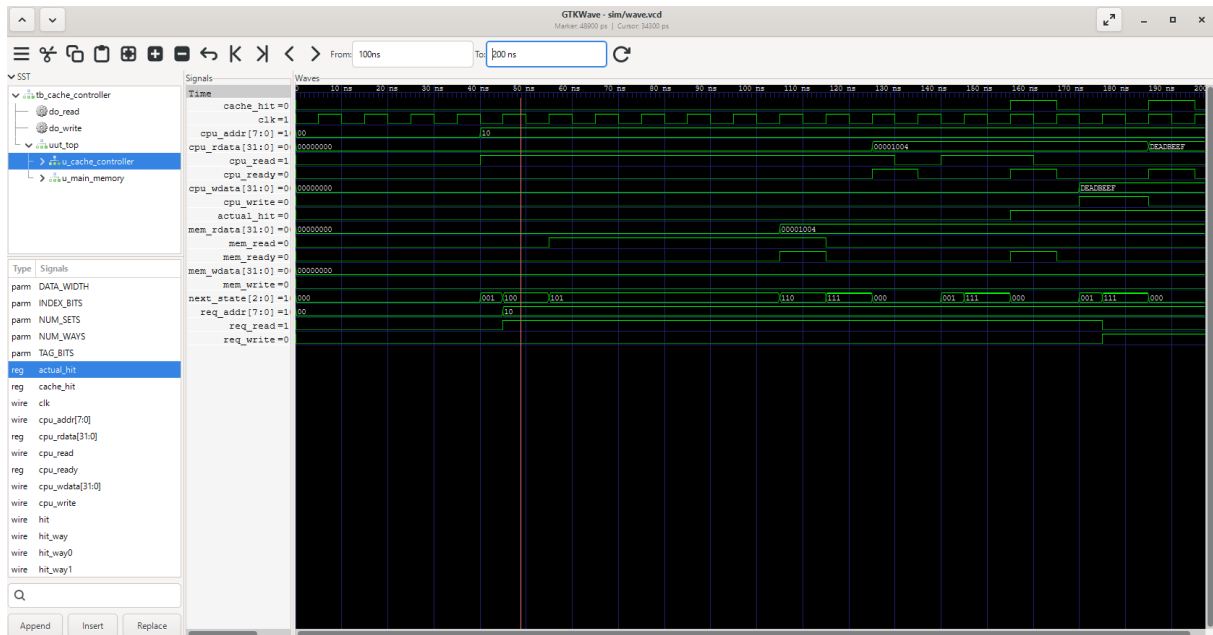


Figura 2: Validação de leitura com miss seguida de leitura com hit para o endereço 8'h10. No primeiro acesso, o controlador ativa `mem_read` e retorna o dado 32'h1004. No segundo acesso ao mesmo endereço, o sinal `cache_hit` é ativado, indicando que o dado já estava presente na cache.

A Figura ?? mostra a validação do caminho de leitura. Inicialmente, a CPU solicita uma leitura no endereço 8'h10. Como a cache estava vazia após o reset, o acesso resulta em miss, indicado por `cache_hit = 0`. O controlador então ativa `mem_read`, aguarda o sinal `mem_ready` da memória principal e retorna o dado 32'h1004 para a CPU.

Em seguida, o mesmo endereço é acessado novamente. Dessa vez, o bloco já está presente na cache, e o sinal `cache_hit` é ativado. O dado é retornado diretamente pela cache, validando o funcionamento correto de uma leitura com hit.

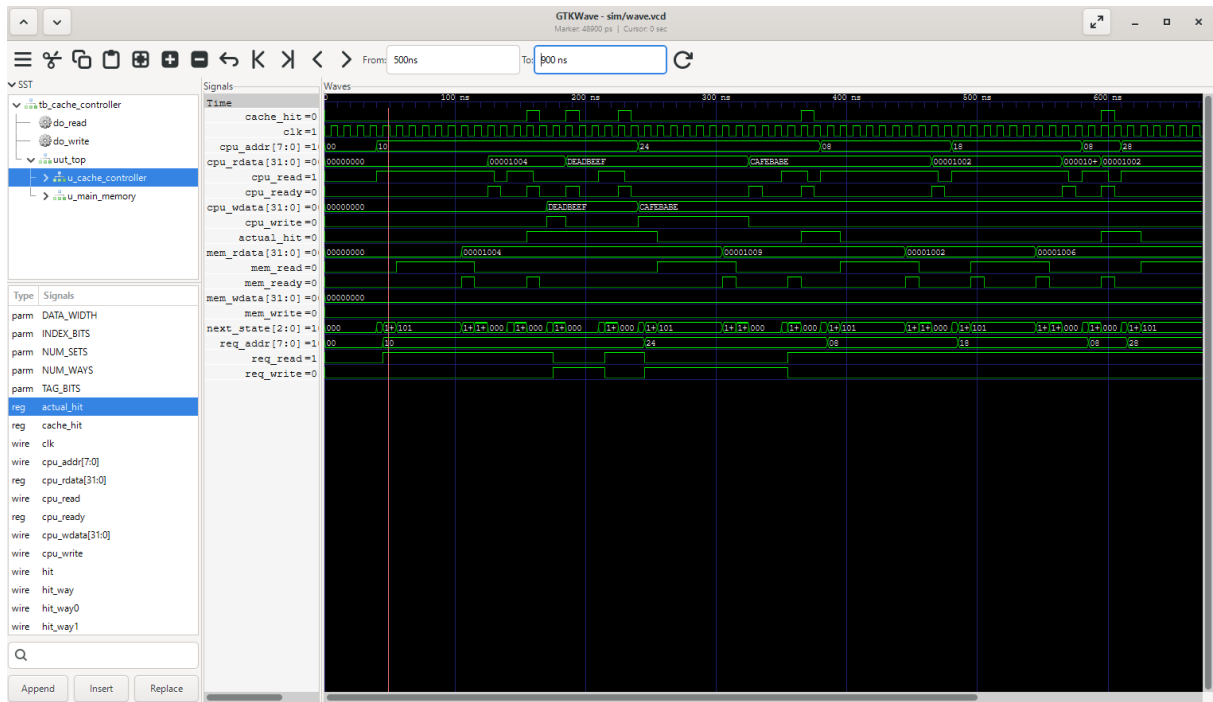


Figura 3: Validação da política de substituição LRU. Os endereços 8'h08, 8'h18 e 8'h28 mapeiam para o mesmo conjunto da cache. Após o novo acesso a 8'h08, esse bloco se torna o mais recentemente usado. Assim, quando 8'h28 é carregado, o bloco correspondente a 8'h18 é substituído.

Na Figura ??, observa-se a sequência de acessos utilizada para validar a política LRU. Primeiro, os endereços 8'h08 e 8'h18 são carregados no mesmo conjunto da cache. Em seguida, o endereço 8'h08 é acessado novamente, atualizando a informação de LRU. Quando o endereço 8'h28, que também mapeia para o mesmo conjunto, é acessado, o controlador substitui o bloco menos recentemente usado, correspondente a 8'h18. Isso é confirmado pelo fato de que uma leitura posterior de 8'h08 resulta em hit, enquanto uma leitura de 8'h18 resulta em miss.

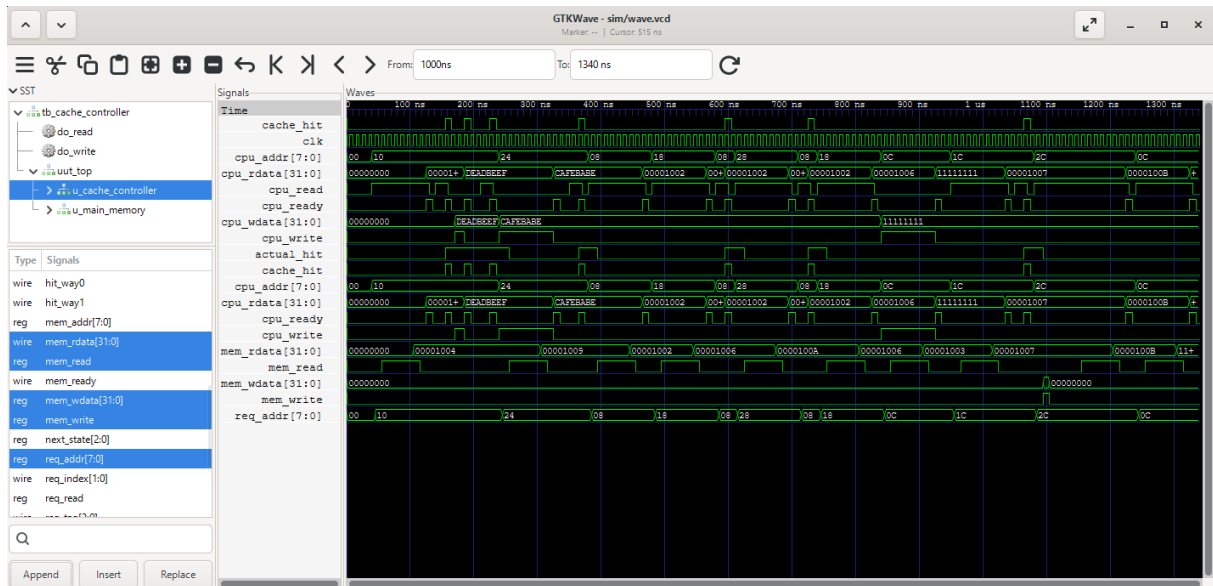


Figura 4: Validação da política *write-back*. A forma de onda mostra a sequência de acessos envolvendo os endereços 8'h0C, 8'h1C e 8'h2C. Após a escrita do valor 32'h11111111 no endereço 8'h0C, o bloco é marcado como *dirty*. Posteriormente, ao ocorrer a substituição desse bloco, o controlador ativa o sinal *mem_write*, escrevendo o valor atualizado de volta na memória principal.

A Figura ?? apresenta a validação da política *write-back*. Inicialmente, ocorre uma escrita no endereço 8'h0C com o valor 32'h11111111. Como a política adotada é *write-back*, esse valor é atualizado na cache e a linha correspondente é marcada como *dirty*, sem escrita imediata na memória principal.

Em seguida, são realizados acessos aos endereços 8'h1C e 8'h2C, que mapeiam para o mesmo conjunto da cache. Esses acessos forçam a substituição do bloco associado ao endereço 8'h0C. Como esse bloco estava marcado como *dirty*, o controlador ativa o sinal *mem_write*, realizando a escrita de volta para a memória principal.

A validação funcional é complementada pelo log da simulação, no qual uma leitura posterior do endereço 8'h0C retorna o valor 32'h11111111. Isso confirma que o dado modificado foi corretamente preservado na memória principal após a substituição do bloco.

5 Documentação do Uso de Inteligência Artificial (Seção 8)

Em conformidade com as diretrizes de transparência do trabalho, os prompts abaixo representam as interações reais realizadas com a IA durante o desenvolvimento do projeto.

5.1 Fase 1: Estruturação e Especificação

Prompt 1:

Envio acima meu trabalho prático de Arquitetura de Computadores III, peço que me explique o que exatamente um controlador de cache precisa e me ajude a estruturar os passos do trabalho.

[Arquivo anexado: Trabalho Prático 1.pdf]

Prompt 2:

Nós faremos com as seguintes especificações:

Cache 2-way set associative, N conjuntos, Política write-back, Política write-allocate, Substituição LRU.

Endereço: 8 bits, Tamanho da palavra: 32 bits, Número de conjuntos: 4,

Associatividade: 2-way, Total de linhas: 8 linhas, Tamanho do bloco: 1 palavra.

5.2 Fase 2: Desenvolvimento do Controlador (cache_controller.sv)

Prompt 3:

perfeito, podemos seguir para a geração em verilog. a ordem de pastas será a que enviei em anexo, gostaria de começar pela cache_controller.sv

já comecei definindo os parâmetros com as informações que te enviei acima
[Trecho de código enviado]

me ajude com o restante do arquivo a partir disso

Prompt 4 (Refinamento de Lógica):

Seguinte, não entendi alguns pontos.

não entendi por que você colocou o offset com 2 bits.

outro problema no miss, ele sempre usa lru_way, mesmo se houver uma via inválida livre. Também falta atualizar o LRU depois de carregar um bloco da memória.

5.3 Fase 3: Desenvolvimento da Memória (main_memory.sv)

Prompt 5:

assim ficou com algumas modificações que fiz [Arquivo cache_controller.sv anexado]. fiz as modificações indicadas, o que acha?

após a aprovação gostaria de iniciar o desenvolvimento da memória principal.

Como o nosso barramento de endereço tem 8 bits, o espaço total de endereçamento é de $2^8 = 256$ bytes, correto?

Prompt 6 (Ajuste de Latência):

como a operação pode demorar um ciclo a mais do que o esperado ao invés de

`delay_counter <= LATENCY ;` coloquei como `delay_counter <= LATENCY - 1;`

além de que a memória não guarda a operação internamente, modifiquei isso também, retirei alguns comentários [Arquivo main_memory.sv anexado]

5.4 Fase 4: Validação e Testes (tb_cache_controller.sv)

Prompt 7:

gostaria de já começarmos as simulações de leitura e escrita

Prompt 8:

como rodar esses testes pelo terminal? estou no windows

Prompt 9:

perfeito, criei um arquivo run.sh e rodando ele obtive os resultados:
[Log do terminal enviado provando o funcionamento]

Prompt 10:

seria algo dessa forma? fiz todos os testes necessários no arquivo em anexo,
valide por favor [Arquivo tb_cache_controller.sv anexado com os testes de limite]